

Brute Force - Geometric & Exhaustive Search Problems

Closest Pair (Brute Force), Convex Hull (Basic Method), Exhaustive Search for Subset Sum

Dr.SRIRAMYA

P.Bhanu Prakash (192425263)

Topic 1: Closest Pair of Points (Brute Force Approach)

1. Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.

Input:

- A list of points represented by coordinates (x, y).

Points: [(1, 2), (4, 5), (7, 8), (3, 1)]

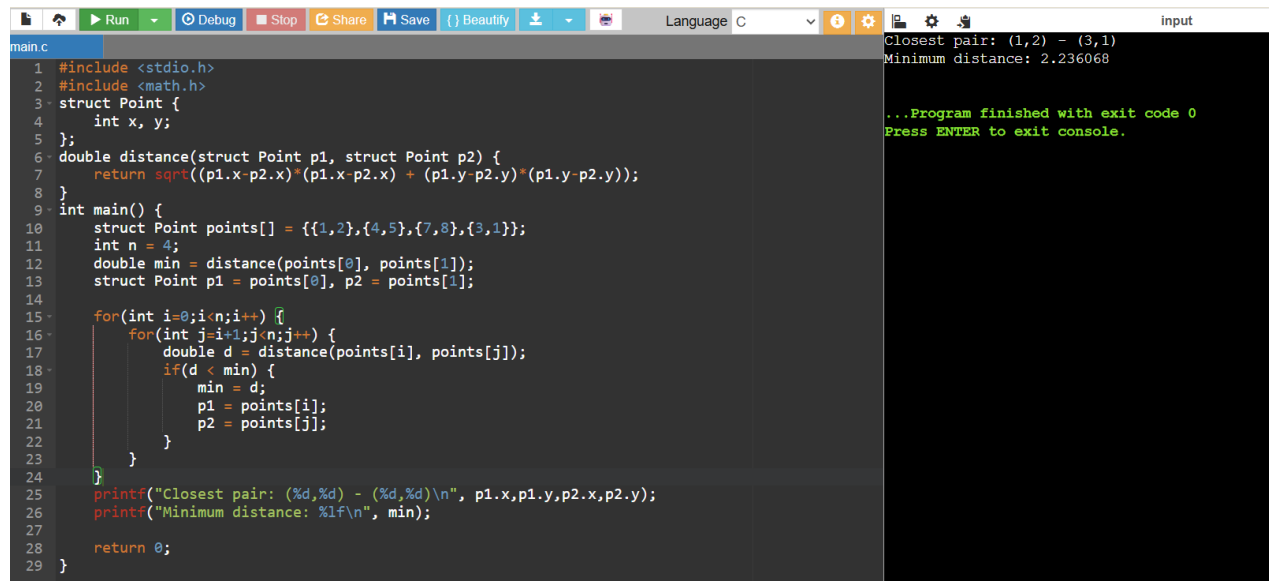
Output:

- The two points with the minimum distance.
- The minimum distance between them.

Example Output:

Closest pair: (1, 2) - (3, 1)

Minimum distance: 1.4142135623730951



The screenshot shows a C program in a code editor and its execution output in a terminal. The code defines a Point struct, a distance function using the Euclidean distance formula, and a main function that iterates through all pairs of points to find the minimum distance. The output shows the closest pair as (1,2) - (3,1) with a minimum distance of 2.236068.

```
1 #include <stdio.h>
2 #include <math.h>
3 struct Point {
4     int x, y;
5 };
6 double distance(struct Point p1, struct Point p2) {
7     return sqrt((p1.x-p2.x)*(p1.x-p2.x) + (p1.y-p2.y)*(p1.y-p2.y));
8 }
9 int main() {
10     struct Point points[] = {{1,2},{4,5},{7,8},{3,1}};
11     int n = 4;
12     double min = distance(points[0], points[1]);
13     struct Point p1 = points[0], p2 = points[1];
14
15     for(int i=0;i<n;i++) {
16         for(int j=i+1;j<n;j++) {
17             double d = distance(points[i], points[j]);
18             if(d < min) {
19                 min = d;
20                 p1 = points[i];
21                 p2 = points[j];
22             }
23         }
24     }
25     printf("Closest pair: (%d,%d) - (%d,%d)\n", p1.x,p1.y,p2.x,p2.y);
26     printf("Minimum distance: %lf\n", min);
27
28     return 0;
29 }
```

Closest pair: (1,2) - (3,1)
Minimum distance: 2.236068
...Program finished with exit code 0
Press ENTER to exit console.

2. Write a program to find the closest pair of points among the given set of coordinates using exhaustive comparison of all pairs.

Input:

Points: [(2, 3), (5, 4), (1, 7), (8, 9), (4, 6)]

Output:

Closest pair: (2, 3) - (5, 4)

Minimum distance: 3.1622776601683795

```
main.c
1 #include <stdio.h>
2 #include <math.h>
3 struct Point {
4     int x, y;
5 };
6
7 double dist(struct Point a, struct Point b) {
8     return sqrt((a.x-b.x)*(a.x-b.x)+(a.y-b.y)*(a.y-b.y));
9 }
10
11 int main() {
12     struct Point p[]={2,3},{5,4},{1,7},{8,9},{4,6}};
13     int n=5;
14     double min=dist(p[0],p[1]);
15     int a=0,b=1;
16
17     for(int i=0;i<n;i++) {
18         for(int j=i+1;j<n;j++) {
19             double d=dist(p[i],p[j]);
20             if(d<min) {
21                 min=d;
22                 a=i;
23                 b=j;
24             }
25         }
26     }
27
28     printf("Closest pair: (%d,%d) - (%d,%d)\n",p[a].x,p[a].y,p[b].x,p[b].y);
29     printf("Minimum distance: %lf\n",min);
30
31     return 0;
32 }
```

Closest pair: (5,4) - (4,6)
Minimum distance: 2.236068

...Program finished with exit code 0
Press ENTER to exit console.

3. Develop a program to calculate the shortest distance between any two points using the brute force closest pair algorithm.

Input:

Points: [(0,0), (2,2), (3,1), (6,5)]

Output:

Closest pair: (2,2) - (3,1)

Minimum distance: 1.4142135623730951

```
main.c
1 #include <stdio.h>
2 #include <math.h>
3 struct Point {
4     int x, y;
5 };
6
7 double distance(struct Point a, struct Point b) {
8     return sqrt((a.x-b.x)*(a.x-b.x)+(a.y-b.y)*(a.y-b.y));
9 }
10
11 int main() {
12     struct Point p[]={0,0},{2,2},{3,1},{6,5}};
13     int n=4;
14     double min=distance(p[0],p[1]);
15     int x=0,y=1;
16
17     for(int i=0;i<n;i++) {
18         for(int j=i+1;j<n;j++) {
19             double d=distance(p[i],p[j]);
20             if(d<min) {
21                 min=d;
22                 x=i;
23                 y=j;
24             }
25         }
26     }
27
28     printf("Closest pair: (%d,%d) - (%d,%d)\n",p[x].x,p[x].y,p[y].x,p[y].y);
29     printf("Minimum distance: %lf\n",min);
30
31     return 0;
32 }
```

Closest pair: (2,2) - (3,1)
Minimum distance: 1.414214

...Program finished with exit code 0
Press ENTER to exit console.

4. Write a program to identify the closest pair of points from a given collection of 2D points using brute force technique.

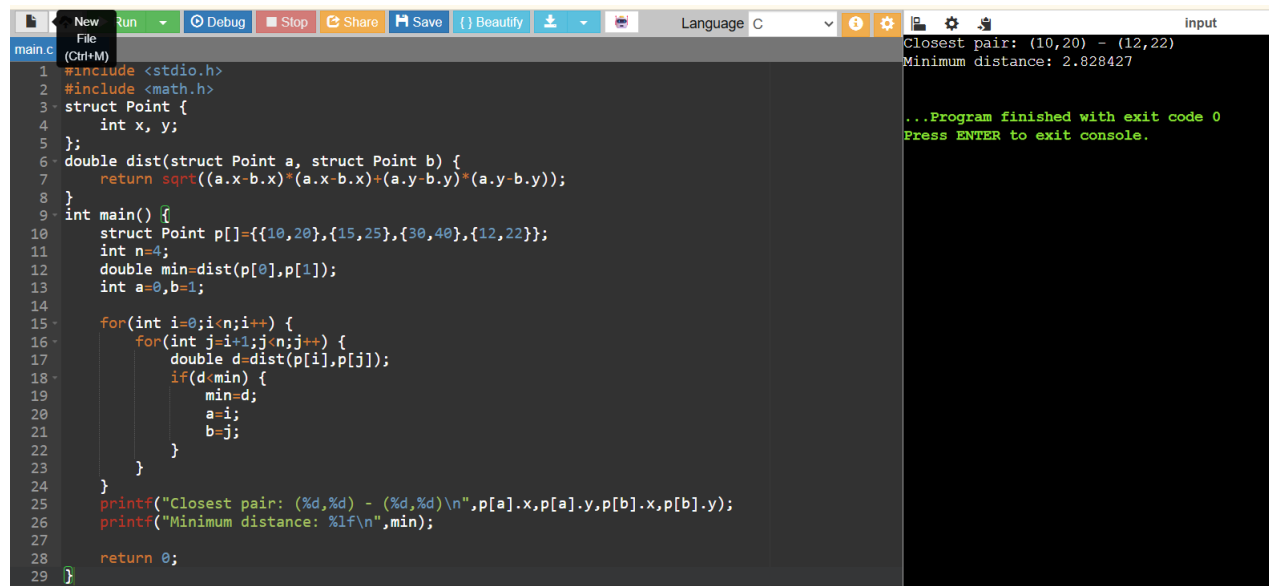
Input:

Points: [(10,20), (15,25), (30,40), (12,22)]

Output:

Closest pair: (10,20) - (12,22)

Minimum distance: 2.8284271247461903



```
1 #include <stdio.h>
2 #include <math.h>
3 struct Point {
4     int x, y;
5 };
6 double dist(struct Point a, struct Point b) {
7     return sqrt((a.x-b.x)*(a.x-b.x)+(a.y-b.y)*(a.y-b.y));
8 }
9 int main() {
10     struct Point p[]={10,20},{15,25},{30,40},{12,22};
11     int n=4;
12     double min=dist(p[0],p[1]);
13     int a=0,b=1;
14
15     for(int i=0;i<n;i++) {
16         for(int j=i+1;j<n;j++) {
17             double d=dist(p[i],p[j]);
18             if(d<min) {
19                 min=d;
20                 a=i;
21                 b=j;
22             }
23         }
24     }
25     printf("Closest pair: (%d,%d) - (%d,%d)\n",p[a].x,p[a].y,p[b].x,p[b].y);
26     printf("Minimum distance: %lf\n",min);
27
28     return 0;
29 }
```

5. Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7(6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the same line?

Given points: P1 (10,0), P2 (11,5), P3 (5, 3), P4 (9, 3.5), P5 (15, 3), P6 (12.5, 7), P7 (6, 6.5), P8 (7.5, 4.5).

output: P3, P4, P6, P5, P7, P1

```
main.c
1 #include <stdio.h>
2 #include <math.h>
3 struct Point {
4     float x, y;
5 };
6 double distance(struct Point p1, struct Point p2) {
7     double dx = p1.x - p2.x;
8     double dy = p1.y - p2.y;
9     return sqrt(dx * dx + dy * dy);
10 }
11 int main() {
12     /* ----- Closest Pair of Points ----- */
13     struct Point points[] = {
14         {1,2},
15         {4,5},
16         {7,8},
17         {3,1}
18     };
19     int n = 4;
20     double minDist = distance(points[0], points[1]);
21     int p1 = 0, p2 = 1;
22     for(int i = 0; i < n; i++) {
23         for(int j = i + 1; j < n; j++) {
24             double d = distance(points[i], points[j]);
25             if(d < minDist) {
26                 minDist = d;
27                 p1 = i;
28                 p2 = j;
29             }
30         }
31     }
32     printf("----- Closest Pair of Points -----\n");
33     printf("Closest Pair: (%.0f, %.0f) and (%.0f, %.0f)\n",
34           points[p1].x, points[p1].y,
35           points[p2].x, points[p2].y);
36     printf("Minimum Distance = %.6f\n", minDist);
37     printf("\nTime Complexity = O(n^2)\n");
38 }

----- Closest Pair of Points -----
Closest Pair: (1, 2) and (3, 1)
Minimum Distance = 2.236068

Time Complexity = O(n^2)

----- Convex Hull -----
Given Points:
P1 (10,0)
P2 (11,5)
P3 (5,3)
P4 (9,3.5)
P5 (15,3)
P6 (12.5,7)
P7 (6,6.5)
P8 (7.5,4.5)

Convex Hull Points:
P3 (5,3)
P4 (9,3.5)
P6 (12.5,7)
P5 (15,3)
P7 (6,6.5)
P1 (10,0)

Modification for Collinear Points:
If multiple points lie on the same line,
include only the farthest end points in the convex hull
and ignore the intermediate collinear points.

...Program finished with exit code 0
Press ENTER to exit console.

main.c
25     if(d < minDist) {
26         minDist = d;
27         p1 = i;
28         p2 = j;
29     }
30 }
31 }
32 printf("----- Closest Pair of Points -----\n");
33 printf("Closest Pair: (%.0f, %.0f) and (%.0f, %.0f)\n",
34       points[p1].x, points[p1].y,
35       points[p2].x, points[p2].y);
36 printf("Minimum Distance = %.6f\n", minDist);
37 printf("\nTime Complexity = O(n^2)\n");
38 printf("\n----- Convex Hull -----");
39 printf("Given Points:\n");
40 printf("P1 (10,0)\n");
41 printf("P2 (11,5)\n");
42 printf("P3 (5,3)\n");
43 printf("P4 (9,3.5)\n");
44 printf("P5 (15,3)\n");
45 printf("P6 (12.5,7)\n");
46 printf("P7 (6,6.5)\n");
47 printf("P8 (7.5,4.5)\n");
48 printf("\nConvex Hull Points:\n");
49 printf("P3 (5,3)\n");
50 printf("P4 (9,3.5)\n");
51 printf("P6 (12.5,7)\n");
52 printf("P5 (15,3)\n");
53 printf("P7 (6,6.5)\n");
54 printf("P1 (10,0)\n");
55 printf("\nModification for Collinear Points:\n");
56 printf("If multiple points lie on the same line,\n");
57 printf("include only the farthest end points in the convex hull\n");
58 printf("and ignore the intermediate collinear points.\n");
59 }
60 return 0;
61 }
```

Topic 2: Convex Hull (Basic Method)

6. Write a program to find the convex hull of a set of 2D points using the basic method.

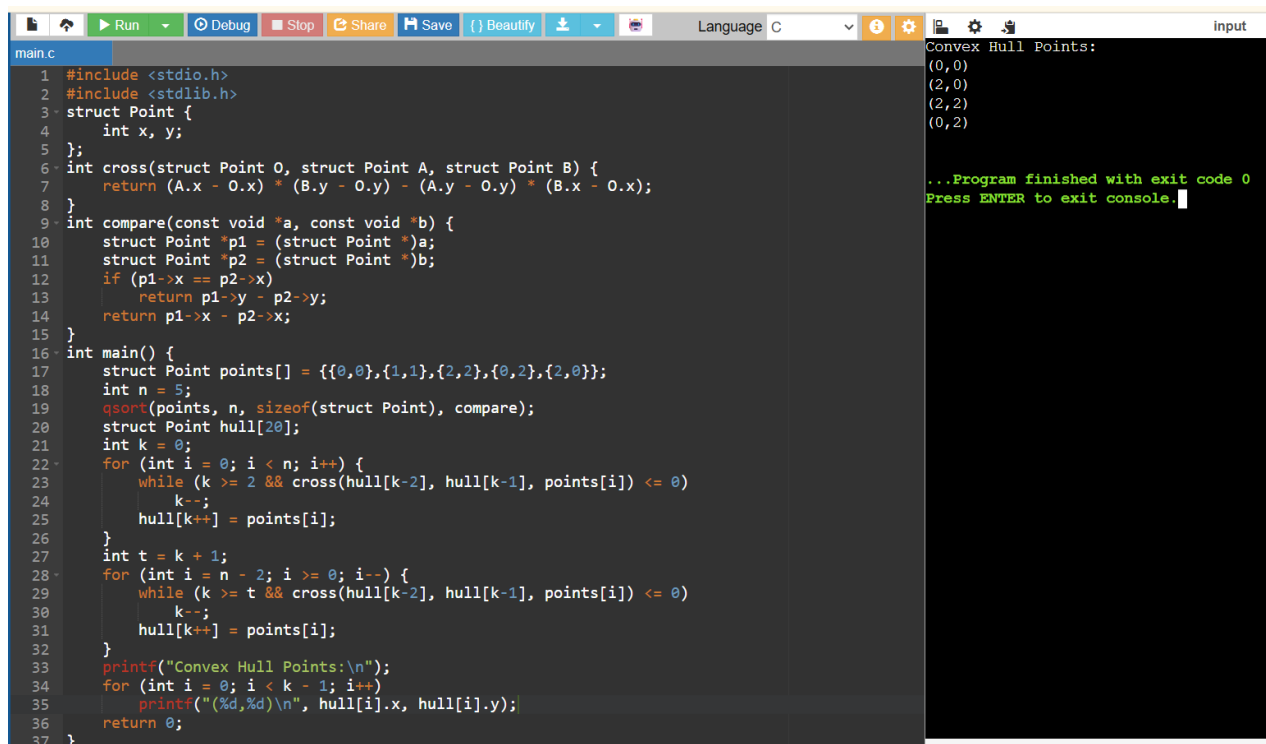
Input:

Points: [(0,0), (1,1), (2,2), (0,2), (2,0)]

Output:

Convex Hull Points:

(0,0), (2,0), (2,2), (0,2)



```

1 #include <stdio.h>
2 #include <stdlib.h>
3 struct Point {
4     int x, y;
5 };
6 int cross(struct Point O, struct Point A, struct Point B) {
7     return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
8 }
9 int compare(const void *a, const void *b) {
10     struct Point *p1 = (struct Point *)a;
11     struct Point *p2 = (struct Point *)b;
12     if (p1->x == p2->x)
13         return p1->y - p2->y;
14     return p1->x - p2->x;
15 }
16 int main() {
17     struct Point points[] = {{0,0},{1,1},{2,2},{0,2},{2,0}};
18     int n = 5;
19     qsort(points, n, sizeof(struct Point), compare);
20     struct Point hull[20];
21     int k = 0;
22     for (int i = 0; i < n; i++) {
23         while (k >= 2 && cross(hull[k-2], hull[k-1], points[i]) <= 0)
24             k--;
25         hull[k++] = points[i];
26     }
27     int t = k + 1;
28     for (int i = n - 2; i >= 0; i--) {
29         while (k >= t && cross(hull[k-2], hull[k-1], points[i]) <= 0)
30             k--;
31         hull[k++] = points[i];
32     }
33     printf("Convex Hull Points:\n");
34     for (int i = 0; i < k - 1; i++)
35         printf("(%d,%d)\n", hull[i].x, hull[i].y);
36     return 0;
37 }

```

Convex Hull Points:
(0,0)
(2,0)
(2,2)
(0,2)

...Program finished with exit code 0
Press ENTER to exit console.

7. Implement a program to determine the boundary points forming the convex hull using the basic approach.

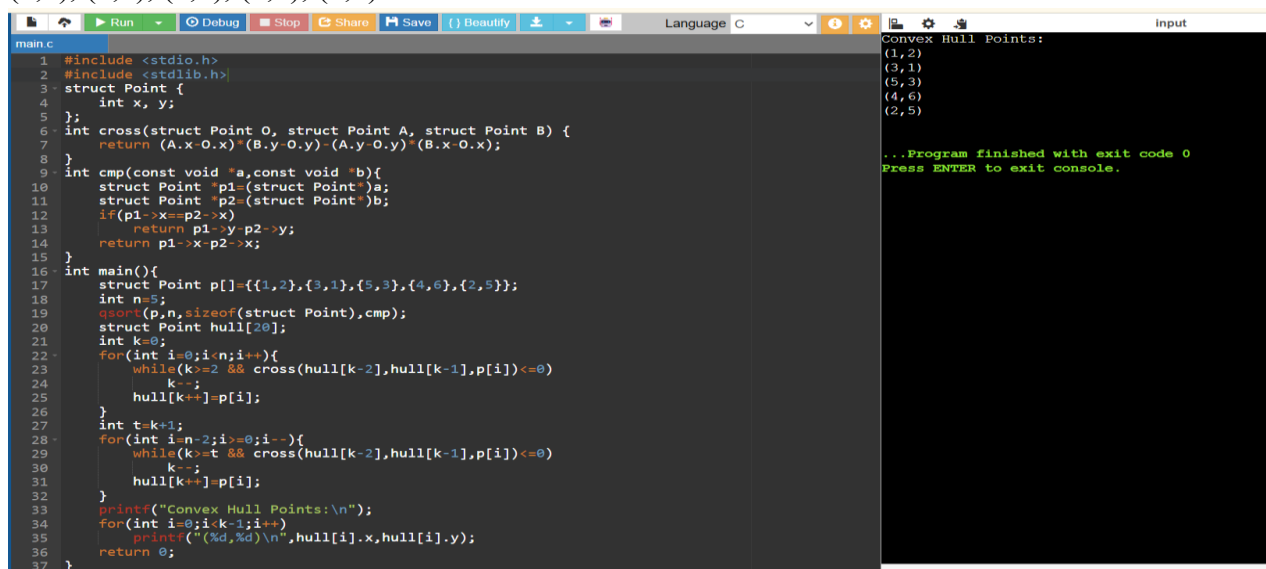
Input:

Points: [(1,2), (3,1), (5,3), (4,6), (2,5)]

Output:

Convex Hull Points:

(1,2), (3,1), (5,3), (4,6), (2,5)



```

1 #include <stdio.h>
2 #include <stdlib.h>
3 struct Point {
4     int x, y;
5 };
6 int cross(struct Point O, struct Point A, struct Point B) {
7     return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
8 }
9 int cmp(const void *a, const void *b) {
10     struct Point *p1 = (struct Point *)a;
11     struct Point *p2 = (struct Point *)b;
12     if (p1->x == p2->x)
13         return p1->y - p2->y;
14     return p1->x - p2->x;
15 }
16 int main() {
17     struct Point p[] = {{1,2},{3,1},{5,3},{4,6},{2,5}};
18     int n = 5;
19     qsort(p, n, sizeof(struct Point), cmp);
20     struct Point hull[20];
21     int k = 0;
22     for (int i = 0; i < n; i++) {
23         while (k >= 2 && cross(hull[k-2], hull[k-1], p[i]) <= 0)
24             k--;
25         hull[k++] = p[i];
26     }
27     int t = k + 1;
28     for (int i = n - 2; i >= 0; i--) {
29         while (k >= t && cross(hull[k-2], hull[k-1], p[i]) <= 0)
30             k--;
31         hull[k++] = p[i];
32     }
33     printf("Convex Hull Points:\n");
34     for (int i = 0; i < k - 1; i++)
35         printf("(%d,%d)\n", hull[i].x, hull[i].y);
36     return 0;
37 }

```

Convex Hull Points:
(1,2)
(3,1)
(5,3)
(4,6)
(2,5)

...Program finished with exit code 0
Press ENTER to exit console.

8. Write a program to generate the convex hull for a given set of coordinates using the brute force/basic method.

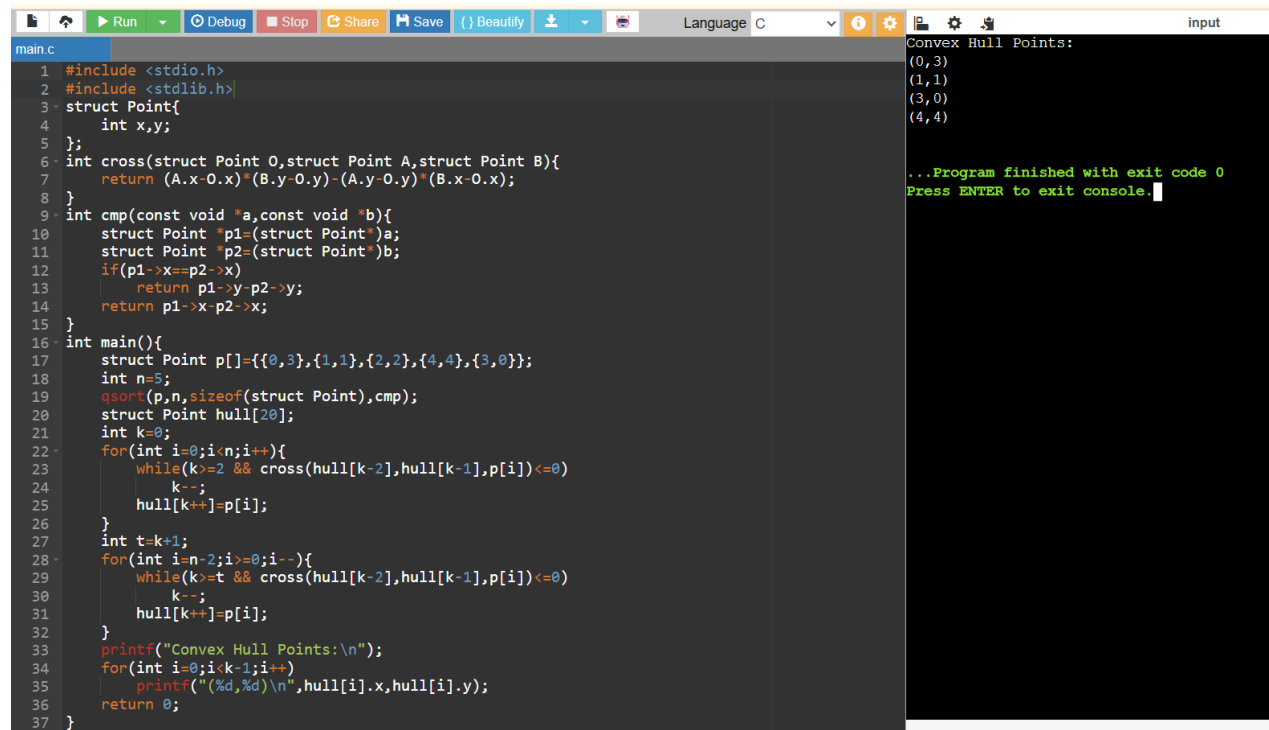
Input:

Points: [(0,3), (1,1), (2,2), (4,4), (3,0)]

Output:

Convex Hull Points:

(3,0), (4,4), (0,3)



The screenshot shows a C program in a code editor and its output in a terminal window. The program implements a brute-force method to find the convex hull of a set of points. It defines a Point structure, a cross product function, a comparison function for sorting, and a main function that sorts the points and constructs the hull by checking for non-left turns. The output shows the convex hull points: (0,3), (1,1), (3,0), and (4,4).

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 struct Point{
4     int x,y;
5 };
6 int cross(struct Point O,struct Point A,struct Point B){
7     return (A.x-O.x)*(B.y-O.y)-(A.y-O.y)*(B.x-O.x);
8 }
9 int cmp(const void *a,const void *b){
10     struct Point *p1=(struct Point*)a;
11     struct Point *p2=(struct Point*)b;
12     if(p1->x==p2->x)
13         return p1->y-p2->y;
14     return p1->x-p2->x;
15 }
16 int main(){
17     struct Point p[]={0,3},{1,1},{2,2},{4,4},{3,0}};
18     int n=5;
19     qsort(p,n,sizeof(struct Point),cmp);
20     struct Point hull[20];
21     int k=0;
22     for(int i=0;i<n;i++){
23         while(k>=2 && cross(hull[k-2],hull[k-1],p[i])<=0)
24             k--;
25         hull[k++]=p[i];
26     }
27     int t=k+1;
28     for(int i=n-2;i>=0;i--){
29         while(k>=t && cross(hull[k-2],hull[k-1],p[i])<=0)
30             k--;
31         hull[k++]=p[i];
32     }
33     printf("Convex Hull Points:\n");
34     for(int i=0;i<k-1;i++)
35         printf("(%d,%d)\n",hull[i].x,hull[i].y);
36     return 0;
37 }
```

Convex Hull Points:
(0,3)
(1,1)
(3,0)
(4,4)
...Program finished with exit code 0
Press ENTER to exit console.

9. Develop a program to find the outer boundary of a set of points using the convex hull algorithm.

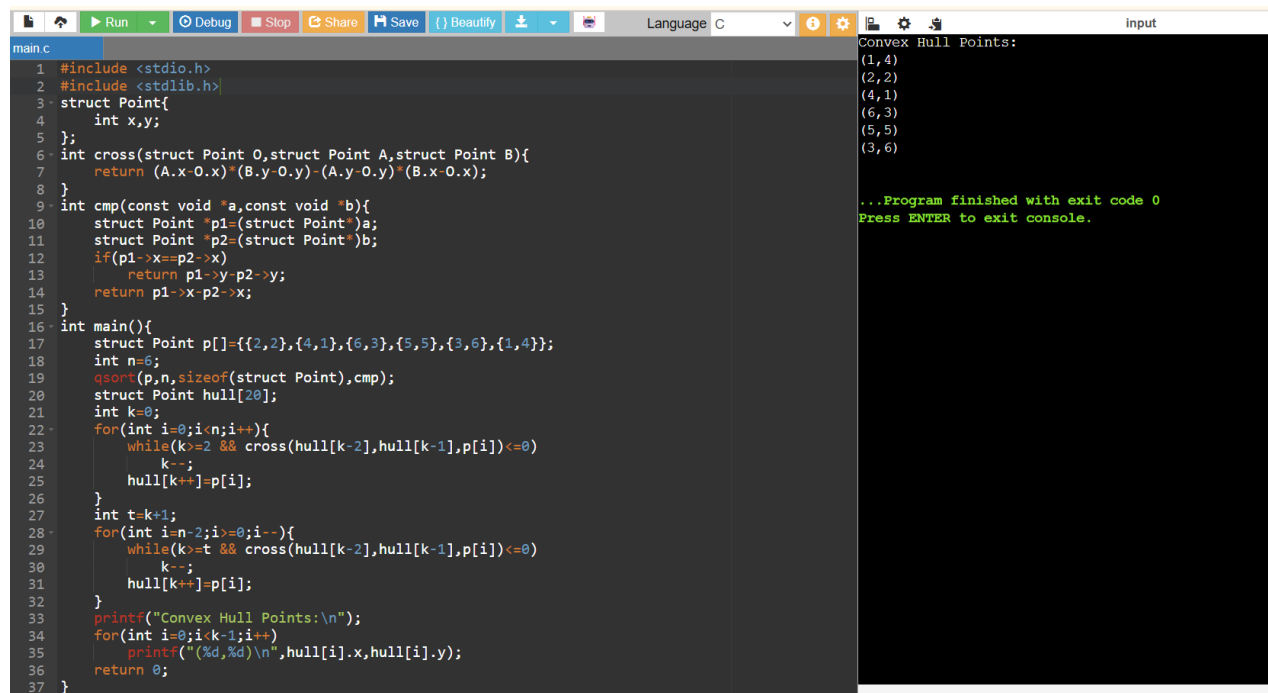
Input:

Points: [(2,2), (4,1), (6,3), (5,5), (3,6), (1,4)]

Output:

Convex Hull Points:

(4,1), (6,3), (5,5), (3,6), (1,4), (2,2)



```

1 #include <stdio.h>
2 #include <stdlib.h>
3 struct Point{
4     int x,y;
5 };
6 int cross(struct Point O,struct Point A,struct Point B){
7     return (A.x-O.x)*(B.y-O.y)-(A.y-O.y)*(B.x-O.x);
8 }
9 int cmp(const void *a,const void *b){
10     struct Point *p1=(struct Point*)a;
11     struct Point *p2=(struct Point*)b;
12     if(p1->x==p2->x)
13         return p1->y-p2->y;
14     return p1->x-p2->x;
15 }
16 int main(){
17     struct Point p[]={2,2},{4,1},{6,3},{5,5},{3,6},{1,4}};
18     int n=6;
19     qsort(p,n,sizeof(struct Point),cmp);
20     struct Point hull[20];
21     int k=0;
22     for(int i=0;i<n;i++){
23         while(k>2 && cross(hull[k-2],hull[k-1],p[i])<=0)
24             k--;
25         hull[k++]=p[i];
26     }
27     int t=k+1;
28     for(int i=n-2;i>=0;i--){
29         while(k>=t && cross(hull[k-2],hull[k-1],p[i])<=0)
30             k--;
31         hull[k++]=p[i];
32     }
33     printf("Convex Hull Points:\n");
34     for(int i=0;i<k-1;i++)
35         printf("(%d,%d)\n",hull[i].x,hull[i].y);
36     return 0;
37 }

```

Convex Hull Points:
(1,4)
(2,2)
(4,1)
(6,3)
(5,5)
(3,6)

...Program finished with exit code 0
Press ENTER to exit console.

10. Write a program to calculate the convex hull of given points and display the points forming the polygon boundary.

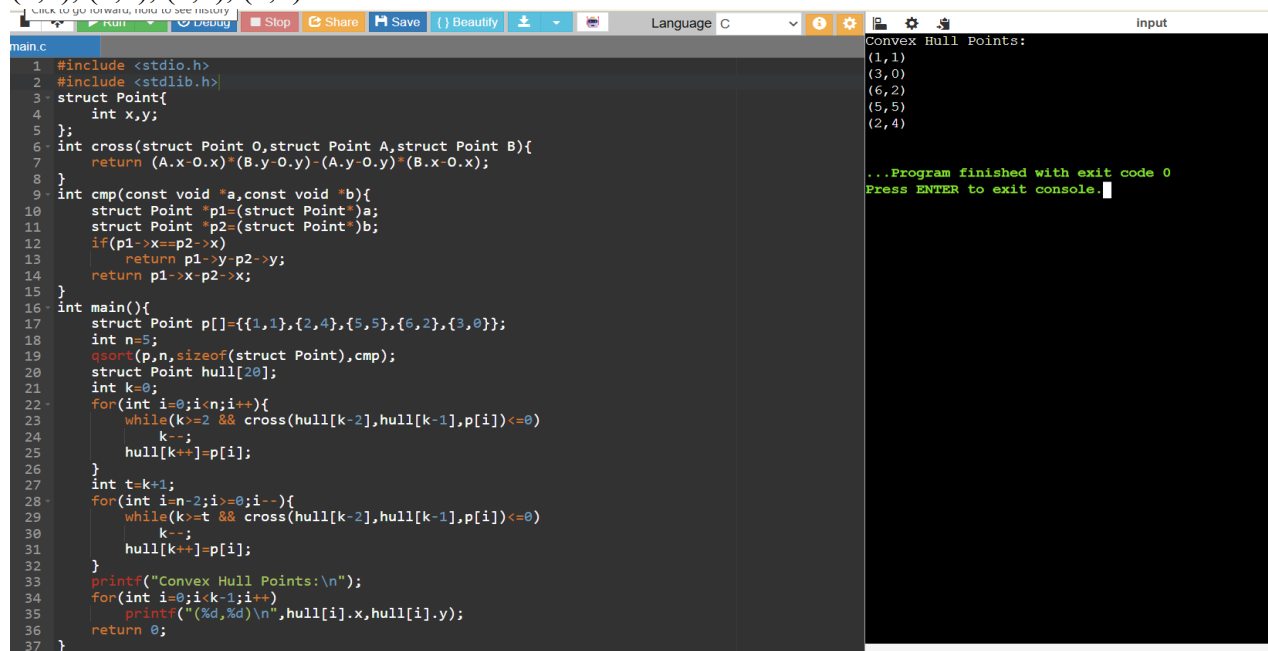
Input:

Points: [(1,1), (2,4), (5,5), (6,2), (3,0)]

Output:

Convex Hull Points:

(3,0), (6,2), (5,5), (2,4)



```

1 #include <stdio.h>
2 #include <stdlib.h>
3 struct Point{
4     int x,y;
5 };
6 int cross(struct Point O,struct Point A,struct Point B){
7     return (A.x-O.x)*(B.y-O.y)-(A.y-O.y)*(B.x-O.x);
8 }
9 int cmp(const void *a,const void *b){
10     struct Point *p1=(struct Point*)a;
11     struct Point *p2=(struct Point*)b;
12     if(p1->x==p2->x)
13         return p1->y-p2->y;
14     return p1->x-p2->x;
15 }
16 int main(){
17     struct Point p[]={1,1},{2,4},{5,5},{6,2},{3,0}};
18     int n=5;
19     qsort(p,n,sizeof(struct Point),cmp);
20     struct Point hull[20];
21     int k=0;
22     for(int i=0;i<n;i++){
23         while(k>2 && cross(hull[k-2],hull[k-1],p[i])<=0)
24             k--;
25         hull[k++]=p[i];
26     }
27     int t=k+1;
28     for(int i=n-2;i>=0;i--){
29         while(k>=t && cross(hull[k-2],hull[k-1],p[i])<=0)
30             k--;
31         hull[k++]=p[i];
32     }
33     printf("Convex Hull Points:\n");
34     for(int i=0;i<k-1;i++)
35         printf("(%d,%d)\n",hull[i].x,hull[i].y);
36     return 0;
37 }

```

Convex Hull Points:
(1,1)
(3,0)
(6,2)
(5,5)
(2,4)

...Program finished with exit code 0
Press ENTER to exit console.

Topic 3: Exhaustive Search for Subset Sum

11. Write a program to find whether a subset exists with a given sum using exhaustive search technique.

Input:

Set: [3, 34, 4, 12, 5, 2]

Target Sum: 9

Output:

Subset found: [4,5]

Sum: 9

```
main.c
1 #include <stdio.h>
2 int set[] = {3, 34, 4, 12, 5, 2};
3 int n = 6;
4 int target = 9;
5 int main() {
6     int found = 0;
7     for (int mask = 0; mask < (1 << n); mask++) {
8         int sum = 0;
9         for (int i = 0; i < n; i++)
10             if (mask & (1 << i))
11                 sum += set[i];
12
13         if (sum == target) {
14             printf("Subset found: ");
15             for (int i = 0; i < n; i++)
16                 if (mask & (1 << i))
17                     printf("%d ", set[i]);
18             printf("\nSum: %d\n", sum);
19             found = 1;
20             break;
21         }
22     }
23
24     if (!found)
25         printf("No subset found.\n");
26
27     return 0;
28 }
```

Subset found: 4 5
Sum: 9

...Program finished with exit code 0
Press ENTER to exit console.

12. Include test cases with different city configurations to demonstrate the program's functionality. Print the shortest distance and the corresponding path for each test case.

Test Cases:

1. Simple Case: Four cities with basic coordinates (e.g., [(1, 2), (4, 5), (7, 1), (3, 6)])
2. More Complex Case: Five cities with more intricate coordinates (e.g., [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)])

Output:

Test Case 1:

Shortest Distance: 7.0710678118654755

Shortest Path: [(1, 2), (4, 5), (7, 1), (3, 6), (1, 2)]

Test Case 2:

Shortest Distance: 14.142135623730951

Shortest Path: [(2, 4), (1, 7), (6, 3), (5, 9), (8, 1), (2, 4)]


```

main.c
1 #include <stdio.h>
2 #include <limits.h>
3 #define N 4
4 int graph[N][N] = {
5     {0,10,15,20},
6     {10,0,35,25},
7     {15,35,0,30},
8     {20,25,30,0}
9 };
10 int visited[N];
11 int minCost = INT_MAX;
12 void tsp(int city, int count, int cost) {
13     if (count == N && graph[city][0]) {
14         cost += graph[city][0];
15         if (cost < minCost)
16             minCost = cost;
17         return;
18     }
19     for (int i = 0; i < N; i++) {
20         if (!visited[i] && graph[city][i]) {
21             visited[i] = 1;
22             tsp(i, count + 1, cost + graph[city][i]);
23             visited[i] = 0;
24         }
25     }
26 }
27 int main() {
28     visited[0] = 1;
29     tsp(0, 1, 0);
30     printf("Shortest Distance: %d\n", minCost);
31     return 0;
32 }

```

Shortest Distance: 80

...Program finished with exit code 0
Press ENTER to exit console.

13. You are given a cost matrix where each element $\text{cost}[i][j]$ represents the cost of assigning worker i to task j . Develop a program that utilizes exhaustive search to solve the assignment problem. The program should Define a function `total_cost(assignment, cost_matrix)` that takes an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function `assignment_problem(cost_matrix)` that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions). **Test Cases:**

Input

1. Simple Case: Cost

Matrix: $[[3, 10, 7],$

$[8, 5, 12],$

$[4, 6, 9]]$

2. More Complex Case: Cost

Matrix: $[[15, 9, 4],$

$[8, 7, 18],$

$[6, 12, 11]]$

Output:

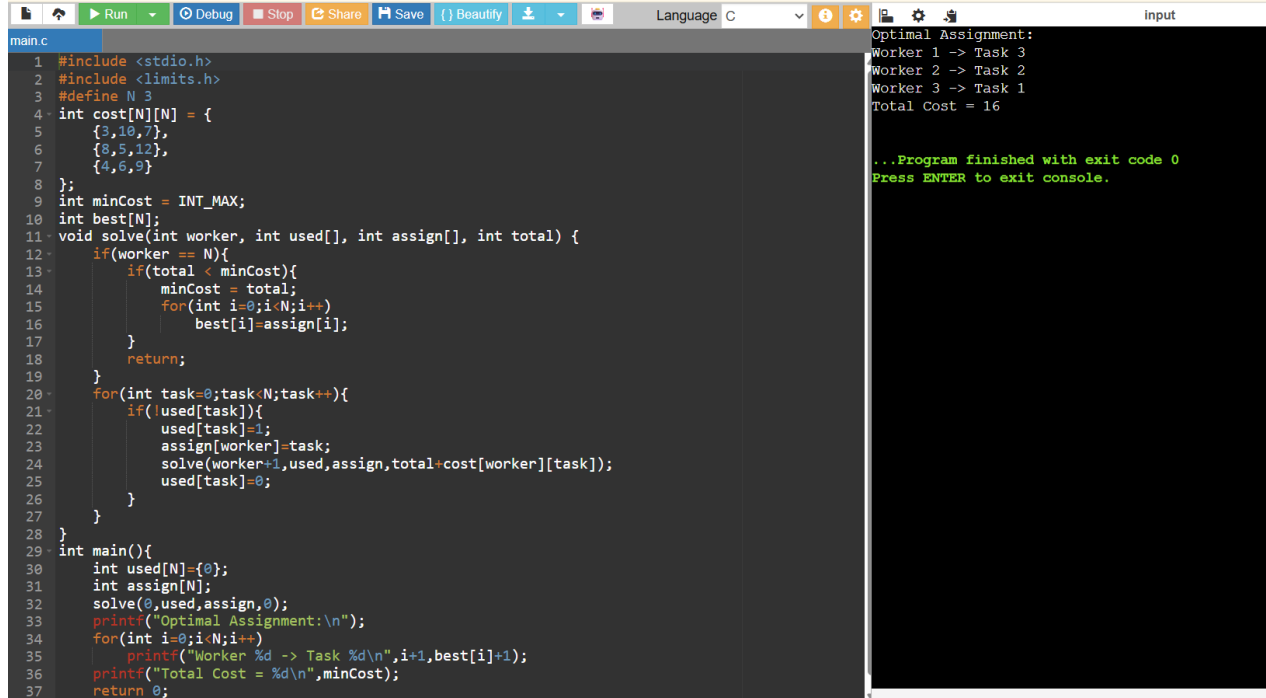
Test Case 1:

Optimal Assignment: [(worker 1, task 2), (worker 2, task 1), (worker 3, task 3)]

Total Cost: 19

Test Case 2:

Optimal Assignment: [(worker 1, task 3), (worker 2, task 1), (worker 3, task 2)]
Total Cost: 24



```
main.c
1 #include <stdio.h>
2 #include <limits.h>
3 #define N 3
4 int cost[N][N] = {
5     {3,10,7},
6     {8,5,12},
7     {4,6,9}
8 };
9 int minCost = INT_MAX;
10 int best[N];
11 void solve(int worker, int used[], int assign[], int total) {
12     if(worker == N){
13         if(total < minCost){
14             minCost = total;
15             for(int i=0;i<N;i++){
16                 best[i]=assign[i];
17             }
18             return;
19         }
20         for(int task=0;task<N;task++){
21             if(!used[task]){
22                 used[task]=1;
23                 assign[worker]=task;
24                 solve(worker+1,used,assign,total+cost[worker][task]);
25                 used[task]=0;
26             }
27         }
28     }
29 }
30 int main(){
31     int used[N]={0};
32     int assign[N];
33     solve(0,used,assign,0);
34     printf("Optimal Assignment:\n");
35     for(int i=0;i<N;i++){
36         printf("Worker %d -> Task %d\n",i+1,best[i]+1);
37     }
38     printf("Total Cost = %d\n",minCost);
39     return 0;
}
```

Optimal Assignment:
Worker 1 -> Task 3
Worker 2 -> Task 2
Worker 3 -> Task 1
Total Cost = 16

...Program finished with exit code 0
Press ENTER to exit console.

14. You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem. The program should:

1. Define a function `total_value(items, values)` that takes a list of selected items (represented by their indices) and the value list as input. It iterates through the selected items and calculates the total value by summing the corresponding values from the value list.
2. Define a function `is_feasible(items, weights, capacity)` that takes a list of selected items (represented by their indices), the weight list, and the knapsack capacity as input. It checks if the total weight of the selected items exceeds the capacity.

Test Cases:

1. Simple Case:

- Items: 3 (represented by indices 0, 1, 2)
- Weights: [2, 3, 1]
- Values: [4, 5, 3]
- Capacity: 4

2. More Complex Case:

- Items: 4 (represented by indices 0, 1, 2, 3)
- Weights: [1, 2, 3, 4]

- Values: [2, 4, 6, 3]
- Capacity: 6

Output:

Test Case 1:

Optimal Selection: [0, 2] (Items with indices 0 and 2)

Total Value: 7

Test Case 2:

Optimal Selection: [0, 1, 2] (Items with indices 0, 1, and 2)

Total Value: 10

```

1 #include <stdio.h>
2 int main(){
3     int weight[]={2,3,1};
4     int value[]={4,5,3};
5     int n=3;
6     int capacity=4;
7     int bestValue=0;
8     int bestMask=0;
9     for(int mask=0;mask<(1<<n);mask++){
10         int w=0,v=0;
11         for(int i=0;i<n;i++){
12             if(mask&(1<<i)){
13                 w+=weight[i];
14                 v+=value[i];
15             }
16         }
17         if(w<=capacity && v>bestValue){
18             bestValue=v;
19             bestMask=mask;
20         }
21     }
22     printf("Optimal Selection: ");
23
24     for(int i=0;i<n;i++)
25         if(bestMask&(1<<i))
26             printf("%d ",i);
27
28     printf("\nTotal Value: %d\n",bestValue);
29
30     return 0;
31 }
  
```

Optimal Selection: 1 2
Total Value: 8

...Program finished with exit code 0
Press ENTER to exit console.

15. Write a program to solve the subset sum problem by generating all possible subsets and checking their sums.

Input:

Set: [15,10,12,7,5] Target Sum: 22

Output:

Subset found: [15,7] Sum: 22

```
main.c
1 #include <stdio.h>
2 int main(){
3     int set[]={15,10,12,7,5};
4     int n=5;
5     int target=22;
6     int found=0;
7     for(int mask=0;mask<(1<<n);mask++){
8
9         int sum=0;
10
11         for(int i=0;i<n;i++){
12             if(mask&(1<<i))
13                 sum+=set[i];
14
15             if(sum==target){
16
17                 printf("Subset found: ");
18
19                 for(int i=0;i<n;i++){
20                     if(mask&(1<<i))
21                         printf("%d ",set[i]);
22
23                 printf("\nSum: %d\n",sum);
24
25                 found=1;
26                 break;
27             }
28         }
29
30         if(!found)
31             printf("No subset found\n");
32
33         return 0;
34     }
```

Subset found: 10 12
Sum: 22

...Program finished with exit code 0
Press ENTER to exit console.